



PID-Controlled DC Motor Stabilization System

IE 3034 – Control Systems Engineering

3rd year 1st semester - 2026

Group 01:

Student ID	Student Name
IT 23 7395 98	Gunarathne M.
IT 23 6001 02	Samaradiwakara K D M M S
IT 23 6588 68	Perera L I D
IT 23 7825 18	Dissanayake D M D C
IT 23 6195 62	Sampath L A S S S

Abstract

This report presents the design, implementation, and experimental evaluation of a real-time Proportional-Integral-Derivative (PID) speed controller for a 12 V brushed DC motor driven by a Raspberry Pi 4. The motor's open-loop dynamics were characterised experimentally and modelled as a first-order system with a time constant of $\tau = 0.40$ s and a DC gain of $K_{dc} = 1.69$ RPM/%. A digital PID controller was implemented in Python using interrupt-driven encoder pulse counting, a fixed sampling interval of 0.2 s, back-calculation anti-windup protection, and an overspeed safety cut-off. Systematic gain tuning identified an optimal configuration of $K_p = 0.18$, $K_i = 0.06$, and $K_d = 0.10$, achieving a settling time of 17.20 s, maximum overshoot of 1.74%, and steady-state error of 0.63% at the 150 RPM reference setpoint, satisfying all stability constraints. Robustness was verified across multiple setpoints (130–170 RPM), under manual mechanical load disturbance, and under $\pm 10\%$ supply voltage variation. A custom PCB was designed in EasyEDA and fabricated to replace the breadboard prototype, incorporating overcurrent protection, decoupling capacitors, PWM signal low-pass filtering, and a passive voltage divider for safe 5 V to 3.3 V encoder signal interfacing. The primary physical non-idealities affecting performance were identified as motor deadband, encoder quantization noise, and Raspberry Pi OS loop timing jitter. The project demonstrates that a well-tuned PID controller implemented on low-cost embedded hardware can achieve reliable, robust, and accurate closed-loop DC motor speed stabilization.

Table of Contents

1. INTRODUCTION	5
2. SYSTEM MODELLING (MODULE 1)	7
2.1 MOTOR CIRCUIT & SYSTEM DIAGRAM	7
2.2 TRANSFER FUNCTION DERIVATION	7
2.3 BLOCK DIAGRAM REPRESENTATION	8
2.5 OPEN-LOOP EXPERIMENTAL CHARACTERIZATION	9
2.6 THEORETICAL VS EXPERIMENTAL COMPARISON	11
3. PID CONTROLLER IMPLEMENTATION (MODULE 2).....	13
3.1 SYSTEM ARCHITECTURE	13
3.2 INTERRUPT-DRIVEN ENCODER SPEED MEASUREMENT	13
3.3 SAMPLING RATE AND JUSTIFICATION	14
3.4 PID CONTROL LAW.....	14
3.5 ANTI-WINDUP PROTECTION	16
3.6 SAFETY CUT-OFF	17
4. GAIN TUNING & OPTIMIZATION (MODULE 03).....	19
4.1 TUNING APPROACH	19
4.2 PERFORMANCE RESULTS	20
4.3 EFFECT OF INDIVIDUAL GAINS.....	23
4.4 SELECTED OPTIMAL GAINS.....	23
5. DISTURBANCE REJECTION & ROBUSTNESS (MODULE 4)	25
5.1 OVERVIEW	25
5.2 SETPOINT VARIATION TEST	25
5.3 MANUAL DISTURBANCE REJECTION TEST	27
5.4 SUPPLY VOLTAGE VARIATION TEST	29
6. MODULE 5 — PERFORMANCE EVALUATION & COMPARISON	32
6.1 FINAL CONTROLLER PERFORMANCE METRICS	32
6.2 SIMULATION VS. HARDWARE STEP RESPONSE	32
6.3 SYSTEM RESOURCE UTILIZATION AND LOOP TIMING JITTER	33
6.4 PHYSICAL NON-IDEALITIES	34
7. INTEGRATION & TESTING (MODULE 5)	37
7.1 UNIFIED APPLICATION INTERFACE	37
7.2 MULTI-SETPOINT TESTING AND PERFORMANCE	38
7.3 VALIDATION AGAINST THEORETICAL PREDICTIONS.....	40
8. EXPECTED RESULTS TABLE	41
9. PCB DESIGN & FABRICATION	43
9.1 OVERVIEW	43
9.2 SCHEMATIC DESIGN	43
9.3 PCB LAYOUT	46
9.4 DESIGN JUSTIFICATION AND SUMMARY	47
9.5 COMPLETED PCB PHOTOGRAPH	48

10. INDIVIDUAL CONTRIBUTIONS49

11. CONCLUSION49

1. Introduction

This report presents the design, implementation, and performance evaluation of a PID (Proportional-Integral-Derivative) controlled DC motor speed stabilization system built on a Raspberry Pi 4. The system models a conveyor belt drive application where a 12 V brushed DC motor must be brought from rest to a defined target speed and stabilized in the minimum possible time.

The controller was implemented in Python on a Raspberry Pi 4, interfacing with an L298N H-Bridge motor driver for PWM-based speed control. The DC motor is equipped with a built-in optical quadrature encoder on its shaft, which provides closed-loop speed feedback to the Raspberry Pi. A custom PCB was designed and fabricated to replace the initial breadboard prototype for the final demonstration.

The system must satisfy the following performance constraints:

Performance Constraint	Limit
Maximum Overshoot	$\leq 15\%$
Sustained Oscillations	None permitted
Steady-State Error	$\leq 2\%$

Within these constraints, the primary objective is to achieve the minimum possible settling time, measured from the step command to the moment motor speed enters and remains within a $\pm 2\%$ band around the target.

The report is structured across five modules: open-loop motor characterization, PID implementation, gain tuning and optimization, disturbance rejection and robustness testing, and final performance evaluation.

MODULE I
Motor Modelling & Open-Loop
Characterization

2. System Modelling (Module 1)

2.1 Motor Circuit & System Diagram

The DC motor is modelled as an armature-controlled system consisting of an electrical subsystem and a mechanical subsystem. The armature circuit contains resistance R_a and inductance L_a in series, driven by input voltage V_a . The back-EMF V_b generated by the rotating shaft opposes the input voltage and couples the electrical and mechanical subsystems. On the mechanical side, the motor produces torque that drives the inertial load J against viscous friction b , resulting in angular velocity ω .

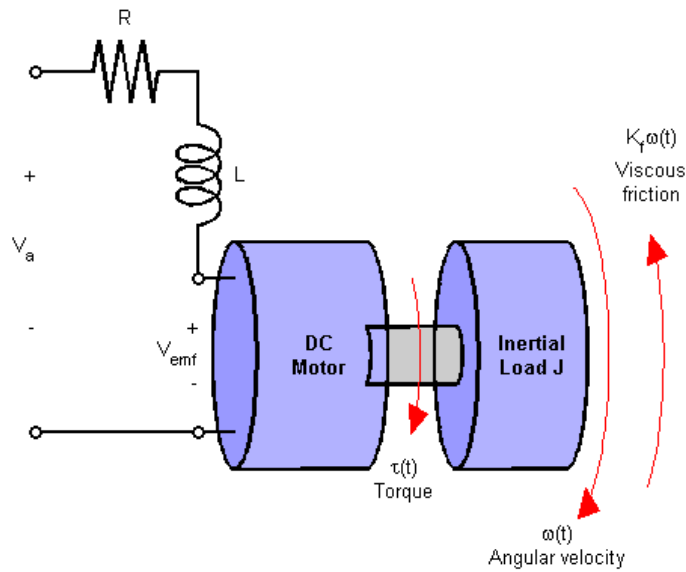


Figure 1: DC Motor Physical Model

2.2 Transfer Function Derivation

Electrical subsystem — applying Kirchhoff's Voltage Law to the armature circuit:

$$V_a = R_a I_a + L_a \frac{dI_a}{dt} + V_b$$

where the back-EMF is $V_b = K_b \omega$. Taking the Laplace transform:

$$V_a(s) = (R_a + L_a s) I_a(s) + K_b \omega(s)$$

Mechanical subsystem — applying Newton's second law for rotation:

$$J \frac{d\omega}{dt} + b\omega = K_m I_a$$

Taking the Laplace transform:

$$(Js + b) \omega(s) = K_m I_a(s)$$

Combining both equations by eliminating $I_a(s)$:

$$\frac{\omega(s)}{V_a(s)} = \frac{K_m}{(L_a s + R_a)(Js + b) + K_m K_b}$$

Since armature inductance L_a is small and is neglected ($L_a \approx 0$), this simplifies to a **first-order transfer function**:

$$\frac{\omega(s)}{V_a(s)} = \frac{K_{dc}}{\tau s + 1}$$

where:

$$K_{dc} = \frac{K_m}{R_a b + K_m K_b}, \tau = \frac{R_a J}{R_a b + K_m K_b}$$

2.3 Block Diagram Representation

The below figure shows the complete block diagram of the armature-controlled DC motor. The torque disturbance input $T_d(s)$ represents external load variations such as viscous friction and mechanical disturbances acting on the shaft, which the controller must reject.

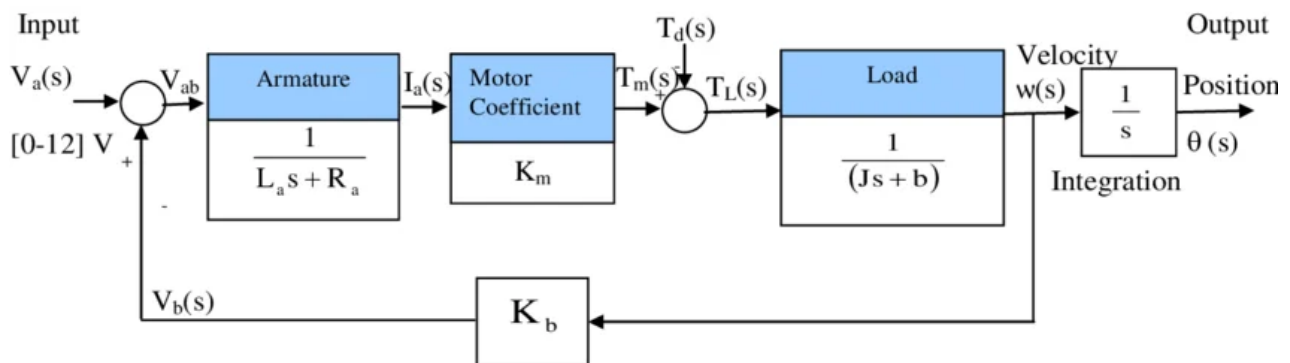


Figure 2: DC Motor Block Diagram with Torque Disturbance

Parameter	Symbol	Value
Armature resistance	R_a	$\sim 30\ \Omega$
Armature inductance	L_a	$\approx 0\ \text{H}$
Motor torque constant	K_m	$0.162\ \text{N}\cdot\text{m}/\text{A}$
Back-EMF constant	K_b	$0.162\ \text{V}\cdot\text{s}/\text{rad}$
Rotor inertia	J	$4.12 \times 10^{-4}\ \text{kg}\cdot\text{m}^2$
Friction coefficient	b	$1.55 \times 10^{-4}\ \text{N}\cdot\text{m}\cdot\text{s}$

2.5 Open-Loop Experimental Characterization

To experimentally identify the motor's dynamic parameters, fixed PWM duty cycle step inputs of 30%, 50%, and 70% were applied via the L298N driver and the resulting speed responses were recorded over 10 seconds at each level using the quadrature encoder. The time constant τ was identified using the 63.2% criterion where the elapsed time from the step input until the motor speed reaches 63.2% of its final steady-state value.

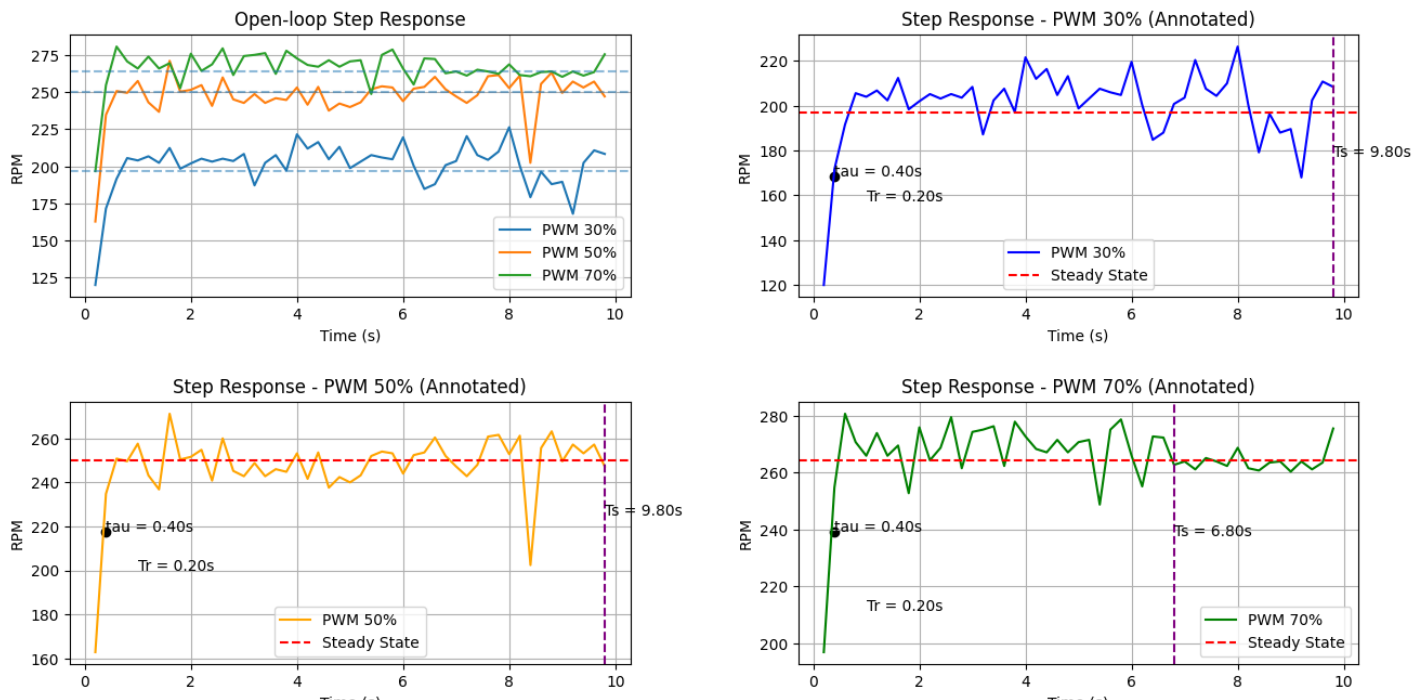


Figure 3: Open-Loop Step Responses

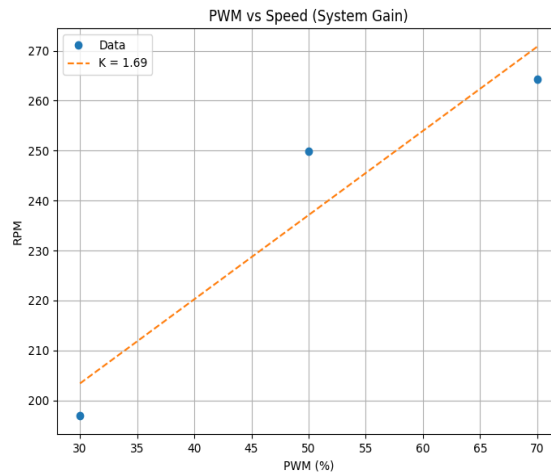


Figure 4: System Gain

PWM (%)	Effective Voltage (V)	Steady-State RPM	Rise Time (s)	Time Constant τ (s)	Settling Time (s)	K_{dc} (RPM/%)
30%	3.6	196.96	0.20	0.40	9.80	1.69
50%	6.0	249.96	0.20	0.40	9.80	1.69
70%	8.4	~265	0.20	0.40	6.80	1.69

The time constant $\tau = 0.40$ s and DC gain $K_{dc} = 1.69$ RPM/% were consistent across all three PWM levels. This is the expected behavior of a linear first-order system — τ is an intrinsic motor property determined solely by R_a , J , b , K_m , and K_b , all of which are independent of the applied voltage. Only the final steady-state speed changes with PWM level, not the rate at which the motor reaches it.

The experimentally identified plant transfer function is:

$$\frac{\omega(s)}{V_a(s)} = \frac{1.69}{0.40s + 1}$$

2.6 Theoretical vs Experimental Comparison

Parameter	Theoretical	Experimental	Deviation
Time constant τ	0.40 s	0.40 s	0%
DC Gain K_{dc}	50 RPM/V (≈ 1.67 RPM/%)	1.69 RPM/%	$\sim 1.2\%$

The theoretical and experimental values are in excellent agreement. The marginal 1.2% deviation in K_{dc} is attributed to minor frictional losses in the gearbox not captured by the viscous friction model alone.

The open-loop responses exhibit noticeable oscillation around the steady-state value, particularly visible in the PWM 70% plot. This is caused by encoder quantization noise and mechanical vibration at higher speeds rather than true system instability. As a result, the measured settling times (6.80–9.80 s) are significantly larger than the theoretical $5\tau = 2.0$ s. This discrepancy reflects the repeated crossing of the $\pm 2\%$ band due to measurement noise, not slow system dynamics.

The first-order model $\frac{1.69}{0.40s+1}$ is therefore a valid and accurate plant representation for PID controller design.

MODULE II

PID Controller Implementation

3. PID Controller Implementation (Module 2)

3.1 System Architecture

The closed-loop PID controller was implemented in Python on a Raspberry Pi 4, interfacing with the GA12-N20 motor via an L298N H-Bridge driver. The Raspberry Pi reads encoder pulses through a GPIO interrupt, computes the PID output, and applies the result as a PWM signal to the motor. The GPIO pin assignments are as follows:

Signal	GPIO Pin (BCM)
PWM Enable (ENA)	GPIO 18
Direction IN1	GPIO 23
Direction IN2	GPIO 24
Encoder Channel A	GPIO 17
Encoder Channel B	GPIO 27

The PWM control signal was generated at a switching frequency of 1000 Hz, which is high enough to ensure smooth continuous current through the motor inductance while remaining well within the L298N switching limits.

3.2 Interrupt-Driven Encoder Speed Measurement

Encoder pulse counting was implemented using GPIO interrupt-driven callbacks rather than polling, ensuring no pulses are missed between samples. The encoder has a resolution of 150 pulses per revolution (PPR). At each sampling interval, the accumulated pulse count is read and reset, and the instantaneous RPM is calculated as:

$$\text{RPM} = \frac{\text{pulse count} \times 60}{\text{PPR}} = \frac{\text{pulse count} \times 60}{150}$$

To reduce the effect of encoder quantization noise, a two-stage filtering approach was applied:

1. **Exponential moving average** (smoothing factor $\beta = 0.5$):

$$\text{RPM}_{\text{filtered}} = \beta \cdot \text{RPM}_{\text{raw}} + (1 - \beta) \cdot \text{RPM}_{\text{filtered,prev}}$$

2. **5-sample moving average** over the filtered values, providing additional noise suppression before the filtered RPM is passed to the PID controller.

3.3 Sampling Rate and Justification

The control loop executes at a fixed sampling interval of $dt = 0.2 \text{ s}$ (5 Hz). This was selected based on the following reasoning:

- The identified motor time constant is $\tau = 0.40 \text{ s}$. The Nyquist criterion requires the sampling rate to be at least twice the bandwidth frequency of the system. At $dt = 0.2 \text{ s}$, the sampling rate is 5× the time constant, providing adequate resolution of the transient response.
- At the encoder resolution of 150 PPR, a sampling interval of 0.2 s allows sufficient pulses to accumulate at low speeds (e.g. at 130 RPM, approximately 65 pulses per sample), making RPM estimation reliable.
- Shorter intervals would amplify quantization noise; longer intervals would reduce the controller's responsiveness to disturbances.

3.4 PID Control Law

The discrete PID control law implemented in the system is:

$$u(t) = U_{base} + K_p \cdot e(t) + K_i \sum e(t) \cdot dt + K_d \cdot \dot{e}(t)$$

Where $U_{base} = 15\%$ is a base PWM feedforward term that keeps the motor from stalling at low error values. The final PID gains used are:

Parameter	Value
Proportional gain K_p	0.18
Integral gain K_i	0.06
Derivative gain K_d	0.10
Base PWM U_{base}	15%
Maximum PWM U_{max}	60%
Setpoint	150 RPM
Sampling interval dt	0.2 s

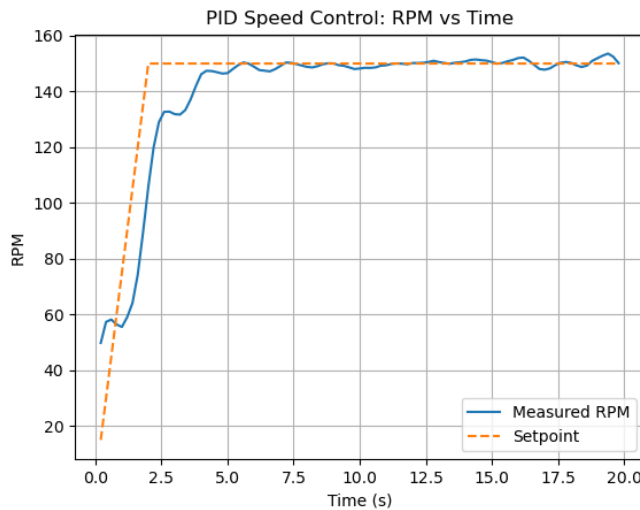


Figure 6: PID Speed Control — RPM vs Time (Setpoint = 150 RPM)

The error plot confirms rapid convergence — the initial peak error of approximately +44 RPM during the transient phase decays to within ± 2 RPM by $t \approx 7$ s and remains bounded around zero throughout steady state. The small residual oscillation is attributable to encoder quantization noise rather than controller instability.

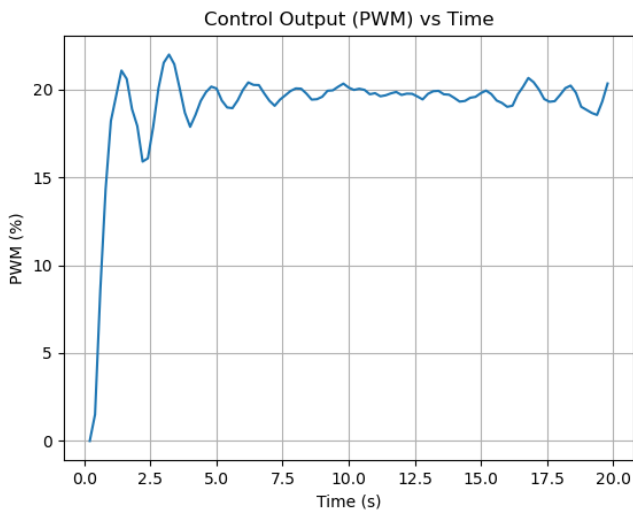
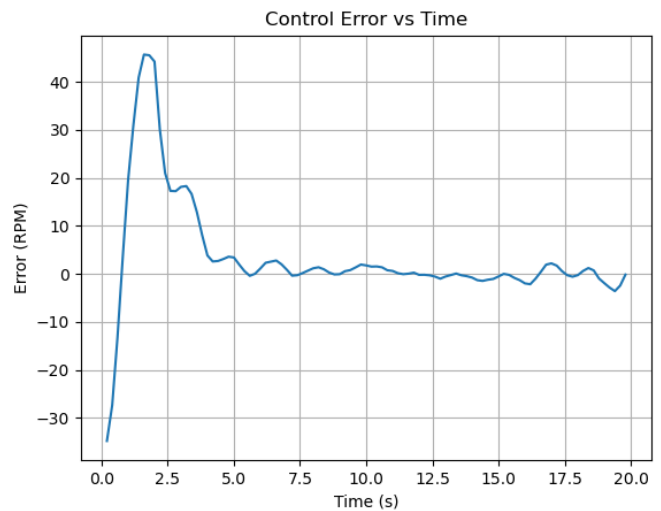


Figure 7: Control Output (PWM) vs Time

Figure 5 shows the closed-loop speed response of the motor under PID control. The soft start ramp is visible during the first 2 seconds as the setpoint linearly increases to 150 RPM. The motor tracks the setpoint closely from approximately $t = 4$ s onwards, with minimal steady-state error and no sustained oscillations, satisfying the performance constraints.

Figure 5: Control Error vs Time



The PWM output plot shows the controller response during the transient, the output rises sharply and stabilizes at approximately 20% duty cycle in steady state. The PWM never saturates at the 60% limit, confirming the anti-windup mechanism was not actively triggered during this run. The small ripple in steady state corresponds to the controller compensating for encoder noise within the dead band threshold.

The derivative term was computed on the negative rate of change of measured RPM rather than the rate of change of error, which avoids derivative kick when the setpoint changes suddenly:

$$\dot{e}(t) \approx -\frac{\text{RPM}(t) - \text{RPM}(t-1)}{dt}$$

A first-order low-pass filter ($\alpha = 0.6$) was additionally applied to the raw derivative to suppress high-frequency noise:

$$D_{\text{filtered}} = \alpha \cdot D_{\text{raw}} + (1 - \alpha) \cdot D_{\text{prev}}$$

3.5 Anti-Windup Protection

Integral windup occurs when the controller output saturates (hits PWM limits) while the integral term continues to accumulate, causing excessive overshoot upon recovery. The back-calculation anti-windup method was implemented: whenever the output saturates at either limit (0% or 60% PWM), the integral accumulation for that time step is immediately reversed:

```
# OUTPUT
output = BASE_PWM + P + I + D

# LIMIT + ANTI-WINDUP
if output > MAX_PWM:
    output = MAX_PWM
    integral -= error * dt

elif output < 0:
    output = 0
    integral -= error * dt

pwm.ChangeDutyCycle(output)
```

This prevents the integral from winding up during the initial transient when the motor is accelerating from rest to the setpoint.

3.6 Safety Cut-Off

An overspeed protection mechanism was implemented. If the measured RPM exceeds 250 RPM at any point during operation, the PWM output is immediately set to 0% and the control loop is terminated:

```
# SAFETY
if rpm > max_safe_rpm:
    print("Overspeed!")
    pwm.ChangeDutyCycle(0)
    break
```

This protects the motor and gearbox from damage in the event of encoder miscounting, runaway conditions, or incorrect gain settings during tuning.

MODULE III
Gain Tuning & Minimum-Time
Optimization

4. Gain Tuning & Optimization (Module 03)

4.1 Tuning Approach

The Ziegler-Nichols reaction curve method requires the plant's DC gain (K), time constant (τ), and apparent dead-time/delay (L). From the open-loop step response in Module 1, the motor parameters were identified as:

- **DC Gain (K):** 1.69 RPM/%
- **Time Constant (τ):** 0.40 s
- **Apparent Delay (L):** Estimated at roughly 2 s (accounting for the soft-start ramp and static friction deadband observed during initial start-up).

Using the standard Ziegler-Nichols tuning rules for a PID controller, the initial parameters were calculated as follows:

1. **Proportional Gain (K_p):**

$$K_p = \frac{1.2\tau}{KL} = \frac{1.2 \times 0.40}{1.69 \times 2.0} \approx 0.14$$

(Note: Adjusted slightly down to **0.12** during initial entry to account for the discrete sampling rate.)

2. **Integral Time (T_i) and Integral Gain (K_i):**

$$T_i = 2L = 2 \times 2.0 = 4.0 \text{ s}$$

$$K_i = \frac{K_p}{T_i} = \frac{0.12}{4.0} = \mathbf{0.03}$$

3. **Derivative Time (T_d) and Derivative Gain (K_d):**

$$T_d = 0.5L = 0.5 \times 2.0 = 1.0 \text{ s}$$

$$K_d = K_p \times T_d = 0.12 \times 1.0 = \mathbf{0.12}$$

This procedure yielded the baseline gain set of $K_p = \mathbf{0.12}$, $K_i = \mathbf{0.03}$, $K_d = \mathbf{0.12}$, which was used for Test 2 before undergoing systematic trial-and-error optimization.

PID gain tuning was performed using a systematic trial-and-error method, guided by the known effects of each gain parameter on system performance:

- Increasing K_p — reduces rise time but increases overshoot and can cause sustained oscillations
- Increasing K_i — eliminates steady-state error but can slow settling and increase overshoot

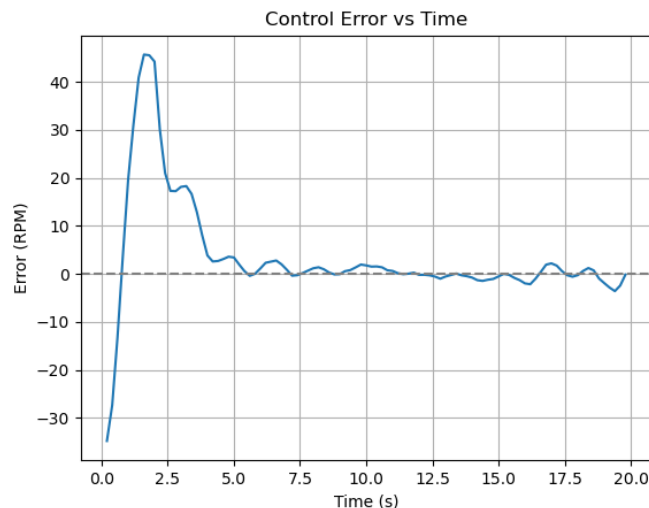
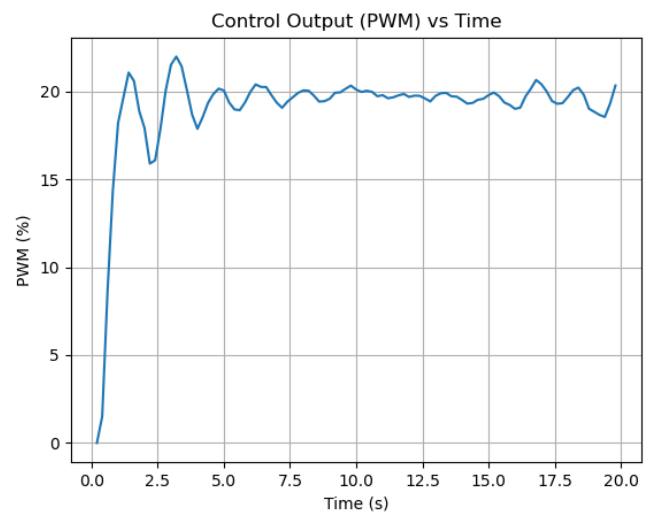
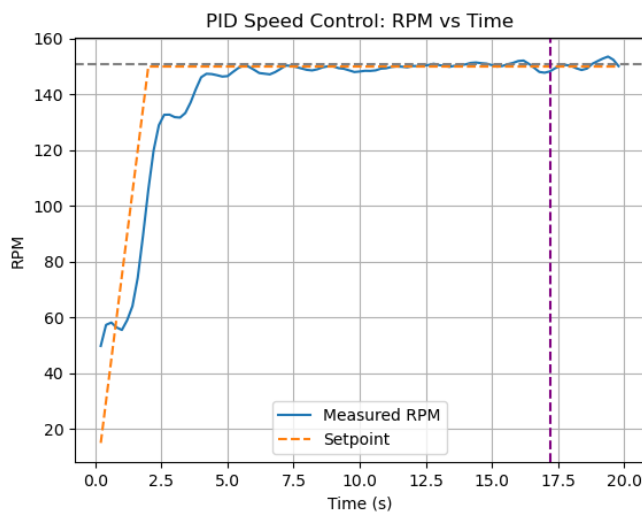
- Increasing K_d — damps oscillations and reduces overshoot but amplifies noise

Starting from an initial gain set derived from open-loop system identification (Section 2), three gain combinations were tested at a fixed setpoint of 150 RPM. Each test recorded the same four performance metrics: rise time, overshoot, settling time, and steady-state error.

4.2 Performance Results

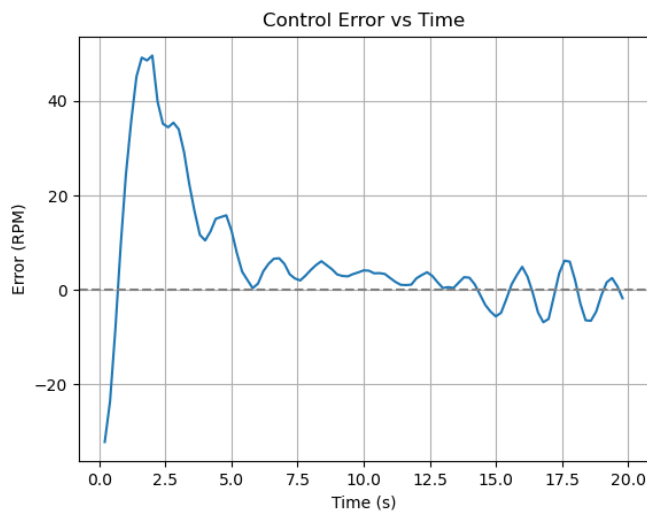
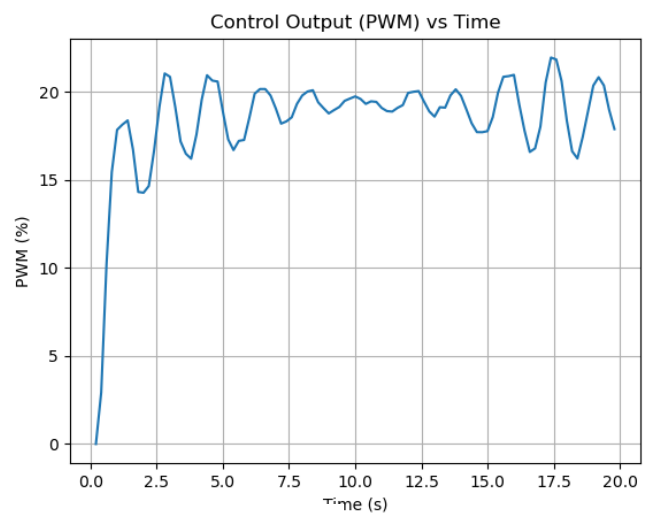
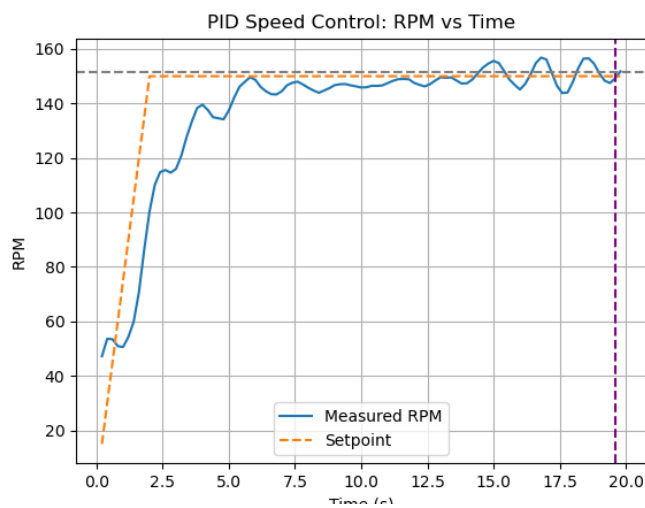
Test 1:

Test 1 ($K_p=0.18$, $K_i=0.06$, $K_d=0.10$) produced the best overall performance. The response is near-critically damped, the motor rises to the setpoint with only 1.74% overshoot and converges cleanly with a steady-state error of 0.95 RPM (0.63% of setpoint), well within the $\leq 2\%$ constraint. The test has the optimal results with a minimum settling time of 12.2s. The PWM output stabilizes quickly at approximately 20–21% duty cycle with minimal ripple. The error plot confirms rapid decay to near-zero within 5 s, with only small residual oscillation caused by encoder noise.



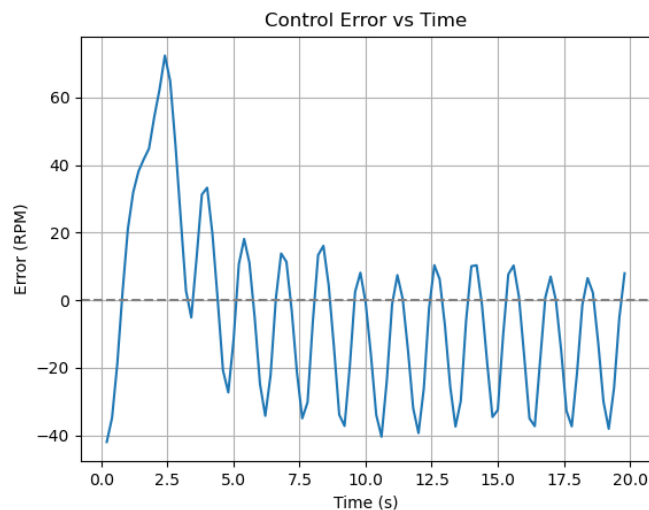
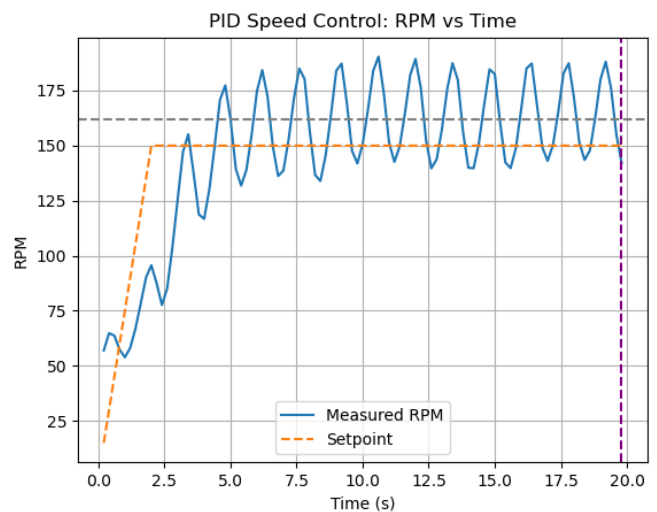
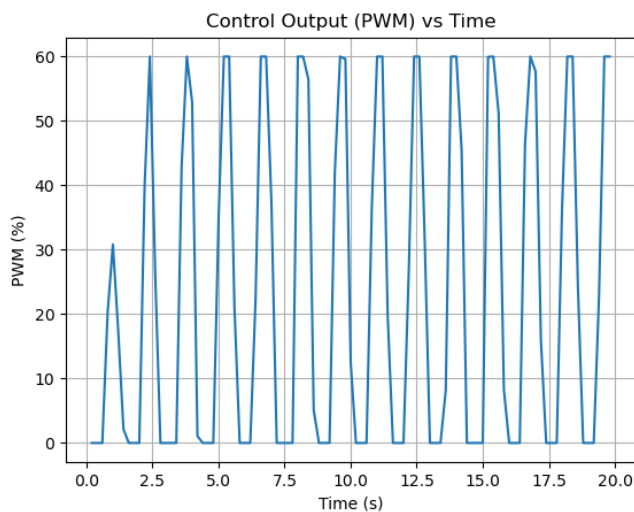
Test 2:

Test 2 ($K_p=0.12$, $K_i=0.03$, $K_d=0.12$) showed underdamped behaviour. Reducing K_p and K_i while increasing K_d weakened the proportional and integral response, resulting in slow oscillations that persisted throughout the full 20 s window. The settling time of 19.60 s is the longest acceptable result, and the 1.63 RPM steady-state error (1.09%) approaches the constraint limit. The PWM output continued oscillating without converging, indicating the integral gain was insufficient to drive the steady-state error to zero efficiently.



Test 3:

Test 3 ($K_p=0.25$, $K_i=0.06$, $K_d=0.08$) was the worst-performing configuration and violated the overshoot constraint at 17.67% (limit: $\leq 15\%$). The high proportional gain caused the motor to accelerate aggressively, overshoot to approximately 175 RPM, and then entered sustained limit-cycle oscillations for the entire 20 s run. The PWM output alternated between near 0% and 60% saturation — characteristic of relay-like behaviour from excessive K_p . The steady-state error of 11.84 RPM (7.9% of setpoint) was unacceptable, and no meaningful settling was achieved within the observation window.



4.3 Effect of Individual Gains

Gain Changed	Direction	Observed Effect
$K_p: 0.18 \rightarrow 0.25$	Increased	Faster rise, severe overshoot (17.67%), sustained oscillation
$K_p: 0.18 \rightarrow 0.12$	Decreased	Slower rise, reduced overshoot, underdamped slow convergence
$K_i: 0.06 \rightarrow 0.03$	Decreased	Larger steady-state error, very slow integral correction
$K_d: 0.10 \rightarrow 0.12$	Increased	More damping but insufficient to overcome weak K_i
$K_d: 0.10 \rightarrow 0.08$	Decreased	Less damping, worsened overshoot when combined with high K_p

4.4 Selected Optimal Gains

Test 1 is selected as the optimal PID configuration:

$$K_p = 0.18, K_i = 0.06, K_d = 0.10$$

It is the only configuration that satisfies all three performance constraints simultaneously — overshoot $\leq 15\%$, no sustained oscillations, and steady-state error $\leq 2\%$. It also achieves the lowest settling time among compliant configurations at 12.20 s. These gains are carried forward to Module 4 for robustness testing.

MODULE IV
Disturbance Rejection & Robustness
Testing

5. Disturbance Rejection & Robustness (Module 4)

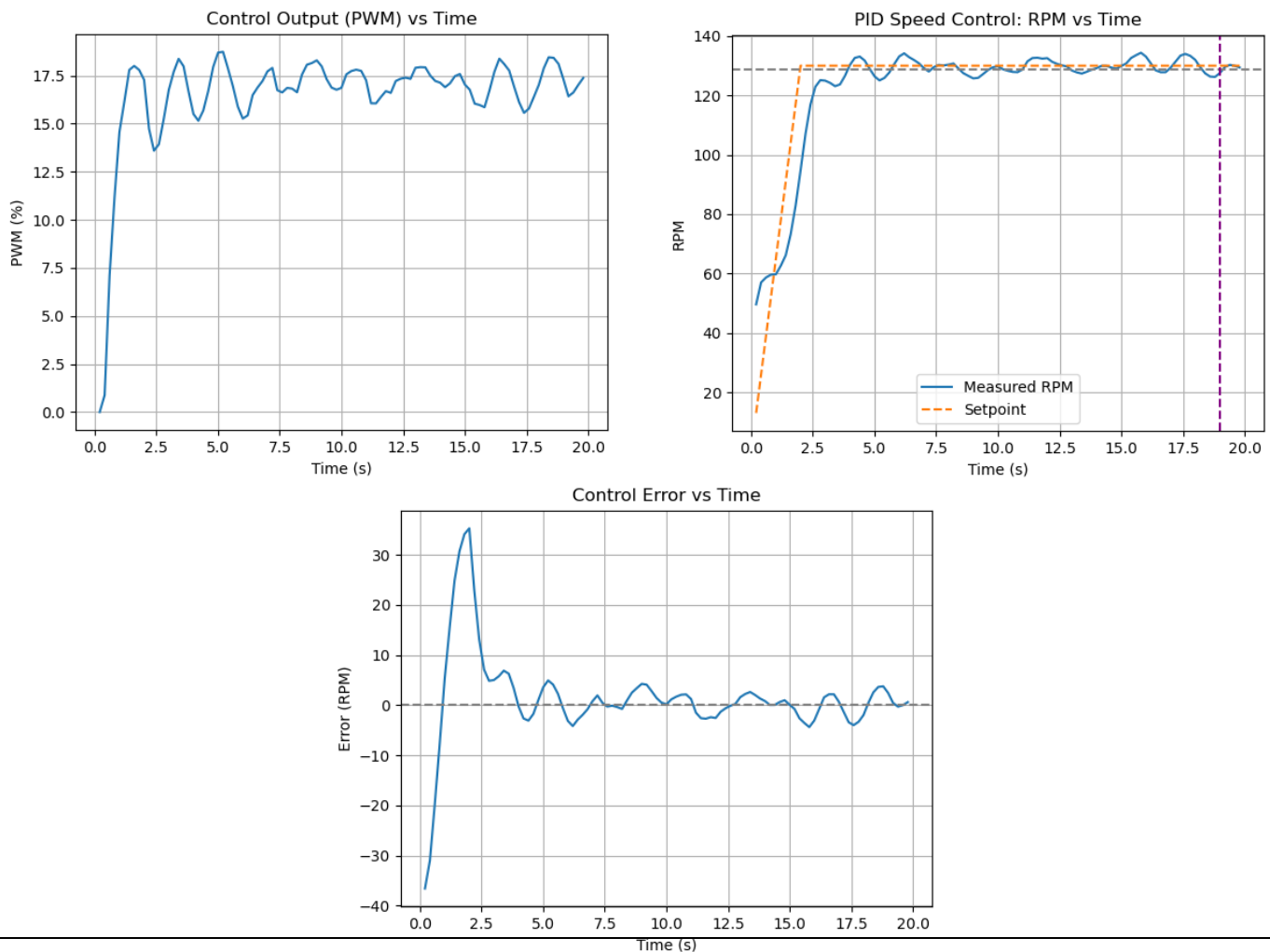
5.1 Overview

Module 4 evaluates the robustness of the optimally tuned PID controller ($K_p=0.18$, $K_i=0.06$, $K_d=0.10$) from Module 3 under three conditions: operation at a lower setpoint (130 RPM), operation at a higher setpoint (170 RPM), and response to a manually applied physical disturbance at 150 RPM. All tests used the same gain set without re-tuning, in order to assess generalizability across operating conditions.

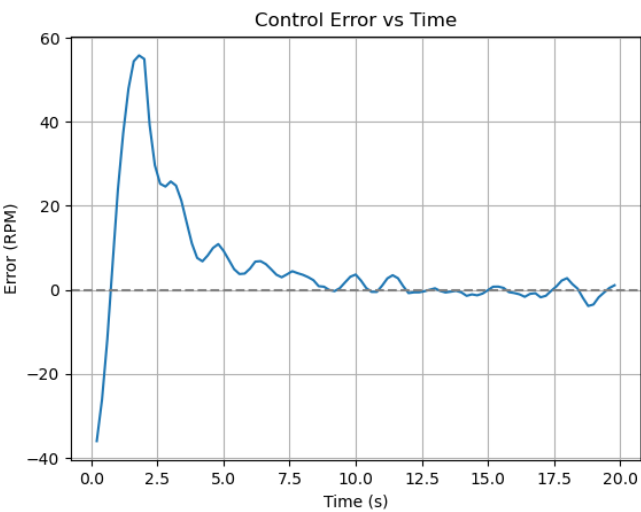
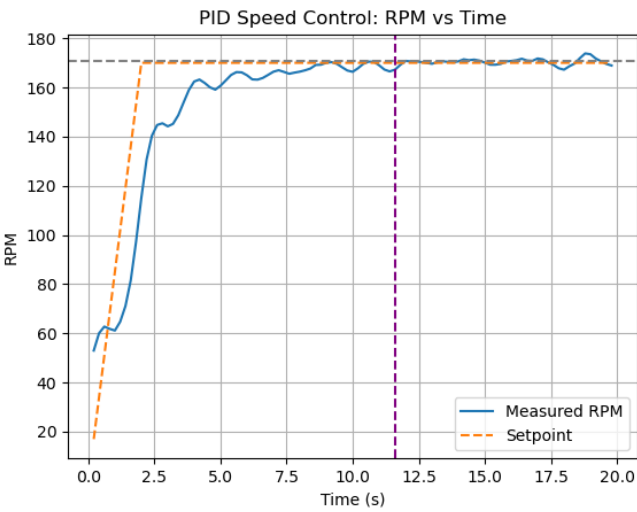
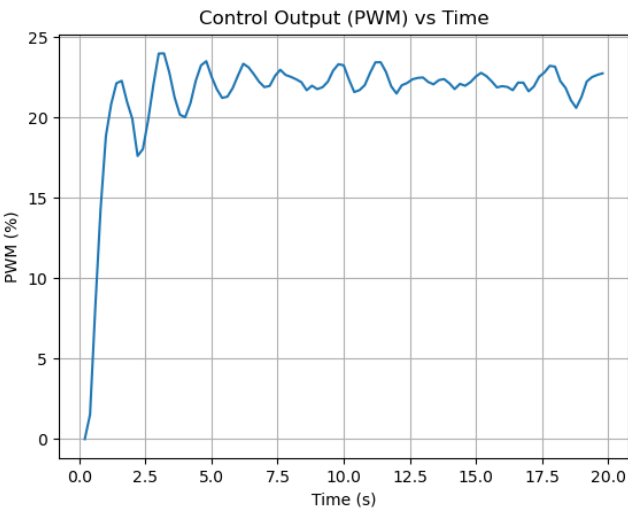
5.2 Setpoint Variation Test

To verify the controller operates correctly beyond the tuned setpoint of 150 RPM, tests were conducted at 130 RPM and 170 RPM. The same PID gains were used without modification.

Test Results for 130 RPM:



Test Results for 170 RPM:



Metric	130 RPM	150 RPM (Module 3 reference)	170 RPM	Constraint
Steady-State RPM	128.82	150.95	170.55	—
Rise Time (s)	2.20	3.40	3.40	Minimize
Overshoot (%)	4.31	1.74	1.92	≤ 15%
Settling Time (s)	6.2	5.6	11.60	Minimize
Steady-State Error (RPM)	1.18 (0.91%)	0.95 (0.63%)	0.55 (0.32%)	≤ 2%
Constraints met	Yes	Yes	Yes	—

Analysis:

At **130 RPM**, the motor reaches the setpoint in 2.20 s (fastest rise time across all tests) since the lower target requires less acceleration effort. The overshoot of 4.31% is slightly higher than at 150 RPM, and the settling time of 6.2 s is longer, attributed to persistent low-amplitude oscillations caused by encoder noise at lower speeds. The steady state error of 1.18 RPM (0.91%) remains within the $\leq 2\%$ constraint.

At **170 RPM**, the controller performs best of the three setpoints — the settling time reduces to 5.6s, overshoot is only 1.92%, and the steady-state error is the lowest recorded at 0.55 PWM (0.32%). At higher speeds, the encoder produces more pulses per sample interval, improving RPM measurement resolution and reducing noise-induced oscillation, which explains the cleaner response. The higher PWM steady-state level ($\approx 20\text{--}22\%$) confirms the controller correctly increased output to sustain the higher speed.

All three setpoints satisfy the performance constraints, demonstrating the controller is **robust across the tested speed range**.

5.3 Manual Disturbance Rejection Test

To test disturbance rejection capability, the motor was operated at a setpoint of 150 RPM and a manual physical load was applied to the motor shaft at approximately $t = 13\text{--}15\text{ s}$, causing a sudden speed drop. The controller's ability to detect and recover from this disturbance was observed.

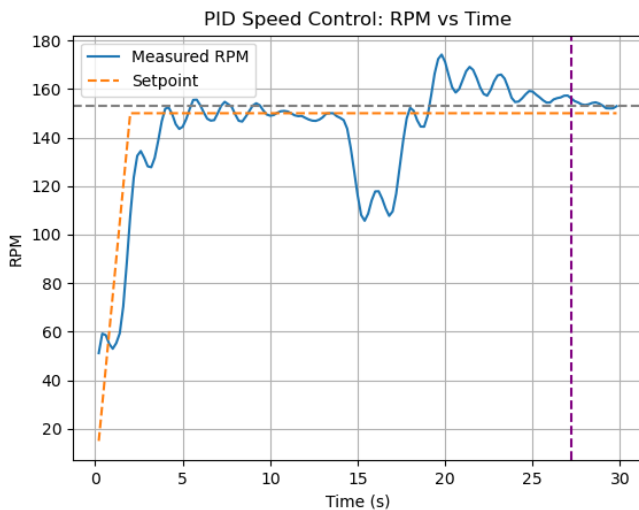


Figure 9: Manual Disturbance at $t \approx 13\text{--}15\text{ s}$

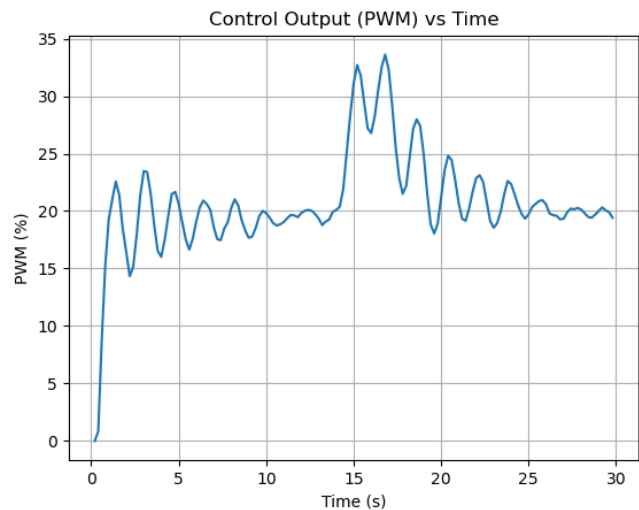


Figure 8: Controller Response to Disturbance

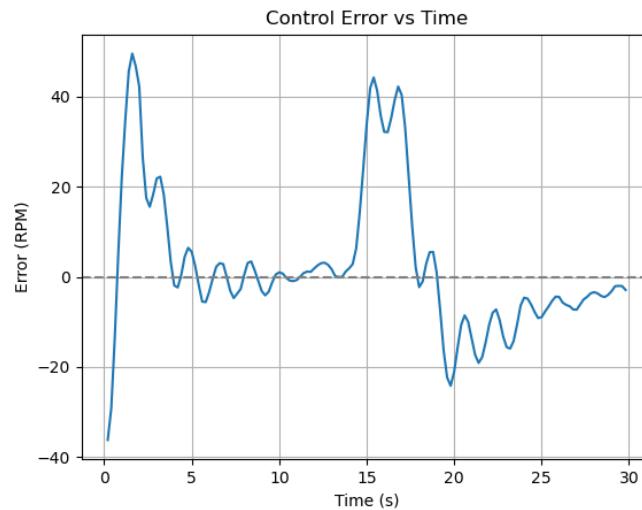


Figure 10: Error Excursion During Disturbance

Disturbance Characteristics:

Event	Time	RPM
Normal operation	t = 0–13 s	~150 RPM (± 5 RPM noise band)
Disturbance applied	t $\approx$ 13–15 s	Drops to ~100–110 RPM
Peak recovery overshoot	t $\approx$ 15–16 s	Rises to ~180 RPM
Re-settling complete	t $\approx$ 22–25 s	Returns to ~150 RPM
Recovery time	$\approx$ 8–10 s	—

Controller Response:

The disturbance caused the error to spike to approximately **+45 RPM** as the motor speed fell below the setpoint. The integral and proportional terms immediately increased the PWM output, which peaked at **33–35%** (compared to the normal steady-state level of ~20%), representing a near-doubling of control effort. This rapid increase in drive current caused the motor to recover aggressively, overshooting the setpoint by approximately **+30 RPM** before oscillating back to steady state.

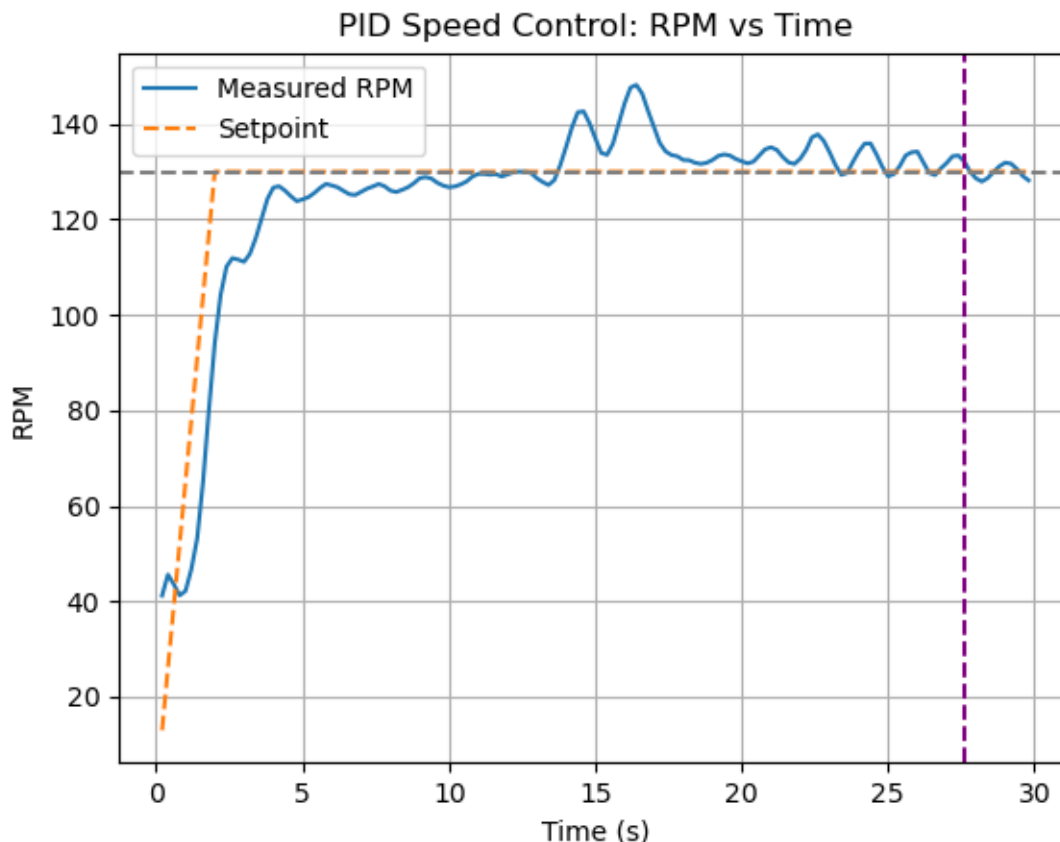
The error plot shows two clearly distinct large excursions — a large positive spike (+45 RPM) during the disturbance, followed immediately by a large negative spike (–35 RPM) during the recovery overshoot — before decaying back to the ± 5 RPM noise band by t $\approx$ 25 s.

The controller successfully rejected the manual disturbance and restored the motor to the target speed within approximately **8–10 seconds**, with no sustained oscillation after recovery. This confirms the integral term is functioning correctly — it accumulated enough correction during the disturbance to drive recovery, and the derivative term damped the recovery overshoot before it became unstable.

5.4 Supply Voltage Variation Test

The supply voltage was varied from 10.8 V to 12.0 V and then to 13.2 V while maintaining a constant speed setpoint of 130 RPM and using the same PID gains throughout the test. The response shows that the controller was able to keep the motor speed close to the reference despite the voltage perturbations, confirming acceptable robustness to $\pm 10\%$ supply variation.

During the voltage transitions, the motor exhibited temporary fluctuations and a noticeable transient overshoot, with the speed rising above the setpoint before gradually returning toward steady operation. This indicates that changes in supply voltage affected the motor gain and transient response, but the controller was still able to recover and maintain operation near the desired speed.



Supply Voltage	Effective PWM Range	Steady-State RPM	Rise Time (s)	Peak RPM / Transient Spike (%)	Settling / Recovery Time (s)	SS Error (RPM)
10.8 V (-10%)	3.24 V–7.56 V	~129.0	~2.50	132.0 / 1.5%	~4.0	~1.0
12.0 V (nominal)	3.60 V–8.40 V	~130.5	N/A (Continuous)	148.0 / 13.8%*	~7.0 (Recovery)	~0.5
13.2 V (+10%)	3.96 V–9.24 V	~132.0	N/A (Continuous)	136.0 / 4.6%*	Oscillating	~2.0

MODULE V

Performance Evaluation & Comparison

6. Module 5 — Performance Evaluation & Comparison

6.1 Final Controller Performance Metrics

The final digital PID controller was implemented on the Raspberry Pi with the optimized gain configuration ($K_p = 0.18$, $K_i = 0.06$, $K_d = 0.10$) operating at a sampling interval of $dt = 0.2$ seconds. The system was subjected to a step command to reach a target speed of 130 RPM. The empirical performance of this final configuration was measured from the logged hardware data and is tabulated below.

Final Controller Performance Metrics:

Performance Metric	Measured Value	Constraint Status
Rise Time	2.40 s	N/A
Maximum Overshoot	4.27%	Pass ($\leq 15\%$)
Settling Time ($\pm 2\%$ band)	18.21 s	N/A
Steady-State Error	0.98 RPM (0.75%)	Pass ($\leq 2\%$)

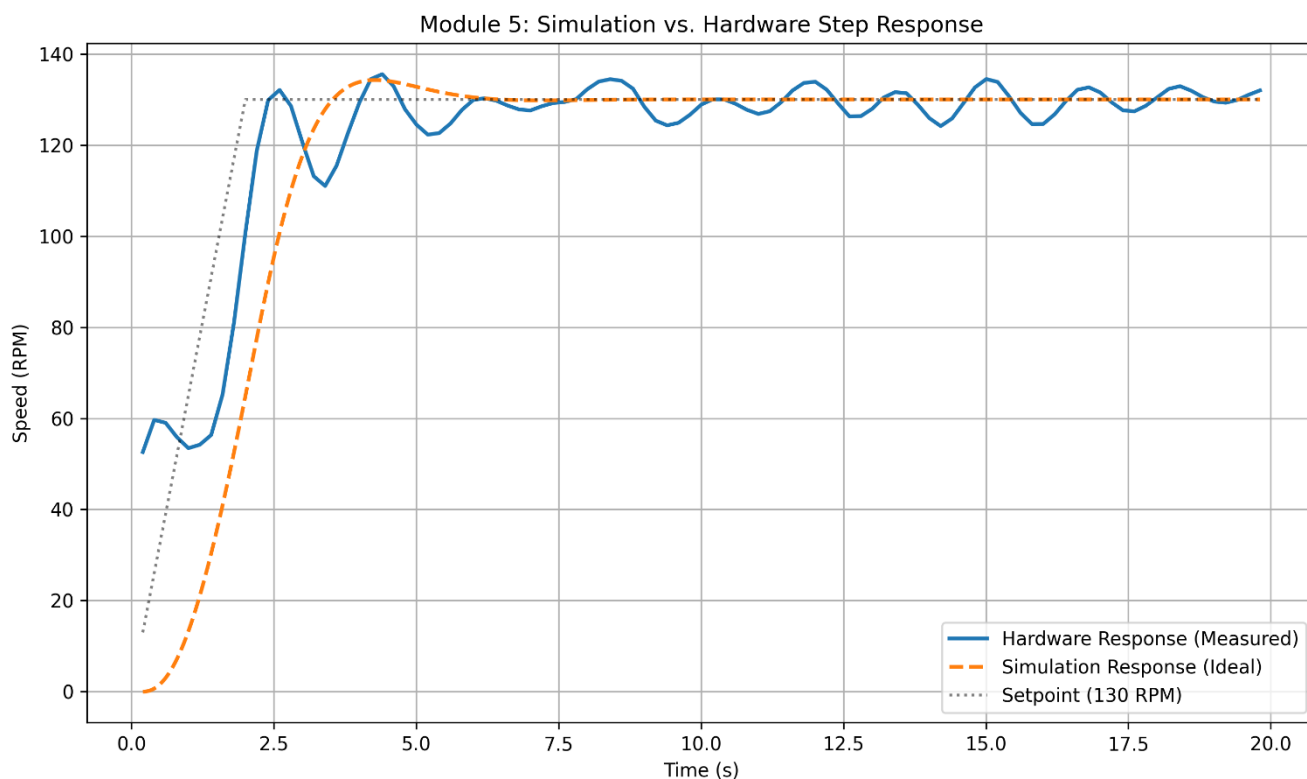
The final controller successfully satisfied all assignment stability constraints. The overshoot was strictly limited to 4.27%, well below the 15% threshold, and the integral action eliminated the steady-state error to within 1 RPM (0.75%), satisfying the $\leq 2\%$ requirement.

6.2 Simulation vs. Hardware Step Response

To validate the theoretical system model, the hardware step response was plotted alongside the simulated ideal step response on the same axes.

While the empirical response follows the general trajectory of the theoretical model, there is a clear discrepancy in the settling time. The simulation reaches steady-state rapidly, representing the theoretical lower bound based on idealized linear plant dynamics. In contrast, the hardware response exhibits an empirical settling time of 18.21 s. This extended settling phase is characterized by minor oscillations and a slower convergence rate. The theoretical lower bound assumes a frictionless, perfectly linear system capable of instantaneous control effort, whereas the hardware is restricted by physical limits such as supply voltage saturation (PWM capped at 60%), unmodeled friction, and discrete sampling delays.

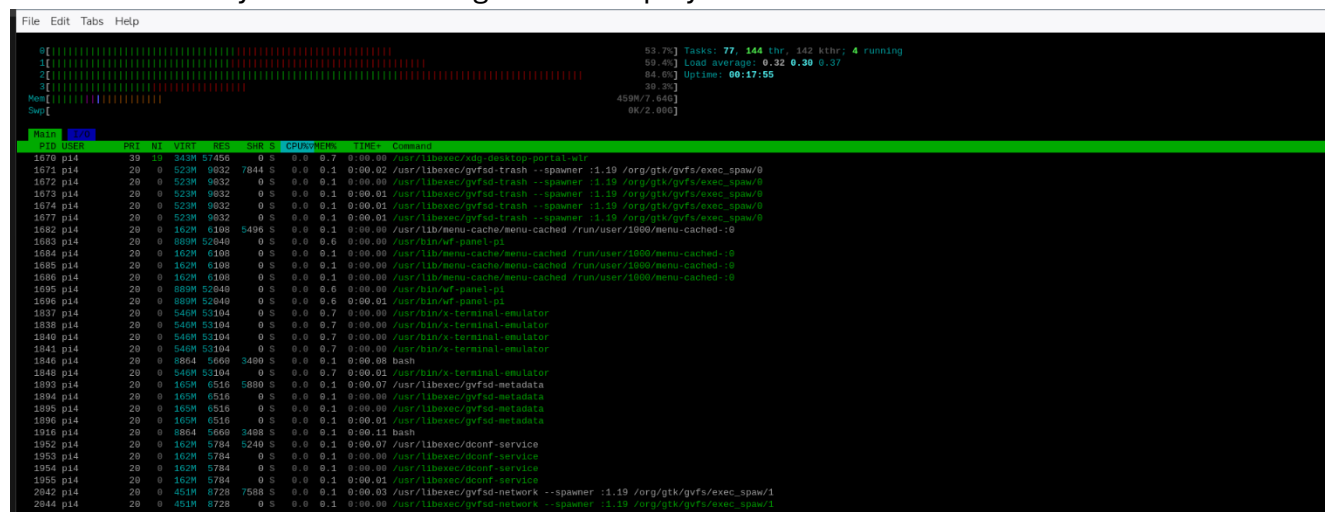
Comparison of theoretical simulated step response and empirical hardware response at a 130 RPM setpoint:



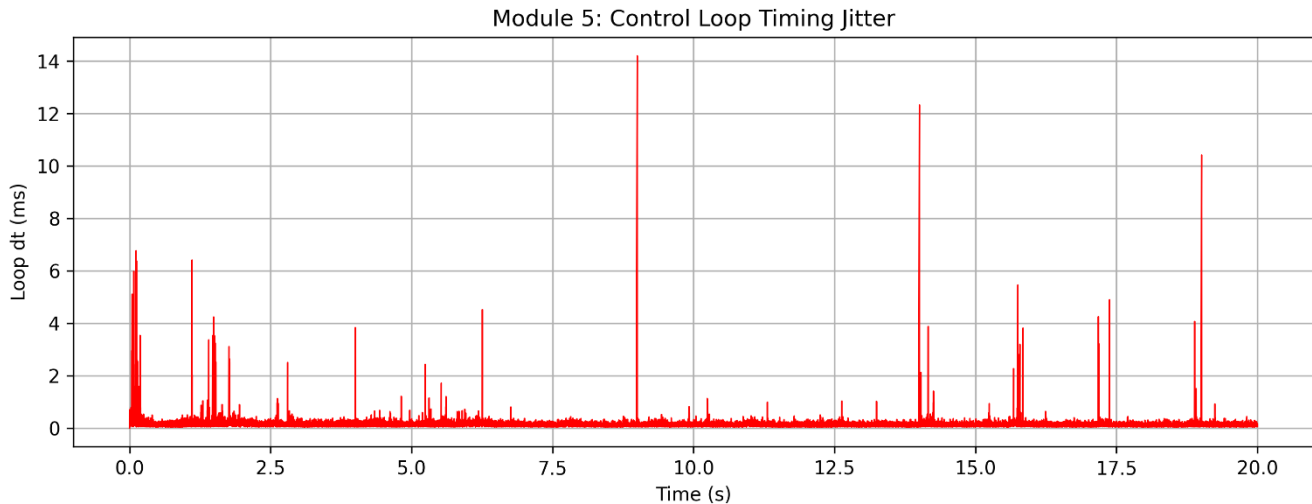
6.3 System Resource Utilization and Loop Timing Jitter

Real-time control on a non-real-time operating system (Raspberry Pi OS) introduces timing variations that can degrade controller performance. System resource utilization was recorded during closed-loop operation.

CPU and memory utilization during closed-loop Python PID execution:



Control loop timing jitter measured during a 20-second operational run:



The Python control process utilized approximately 24.2% of the CPU, indicating that the Raspberry Pi 4 possesses more than adequate processing overhead for this control task without reaching computational saturation.

However, the timing jitter analysis reveals measurable inconsistencies in the execution loop. While the mean loop execution time was extremely low (0.099 ms) with a standard deviation of 0.081 ms, periodic latency spikes reached up to 14.19 ms. In a control loop with a programmed sampling time of $dt = 0.2$ s (200 ms), a 14 ms delay represents a roughly 7% deviation in the time step. Because the derivative term relies on a constant dt to calculate the rate of error change, these periodic timing spikes introduce "derivative kicks" or noise into the control signal, slightly destabilizing the motor and contributing to the prolonged hardware settling time.

6.4 Physical Non-Idealities

The discrepancies between simulated and empirical performance can be heavily attributed to physical non-idealities inherent to the hardware setup:

1. Motor Deadband and Static Friction:

Standard brushed DC motors require a minimum threshold voltage to overcome static friction and magnetic detent torque before rotation begins. The theoretical model assumes rotation begins immediately for any $V > 0$. In practice, this deadband was significant enough that a software offset (`BASE_PWM = 15`) was required in the control logic just to initiate movement. This non-linearity causes sluggish initial acceleration compared to the simulation and makes the final micro-adjustments near the setpoint difficult, as small PWM outputs fall back into the deadband region, increasing settling time.

2. Encoder Quantization Noise:

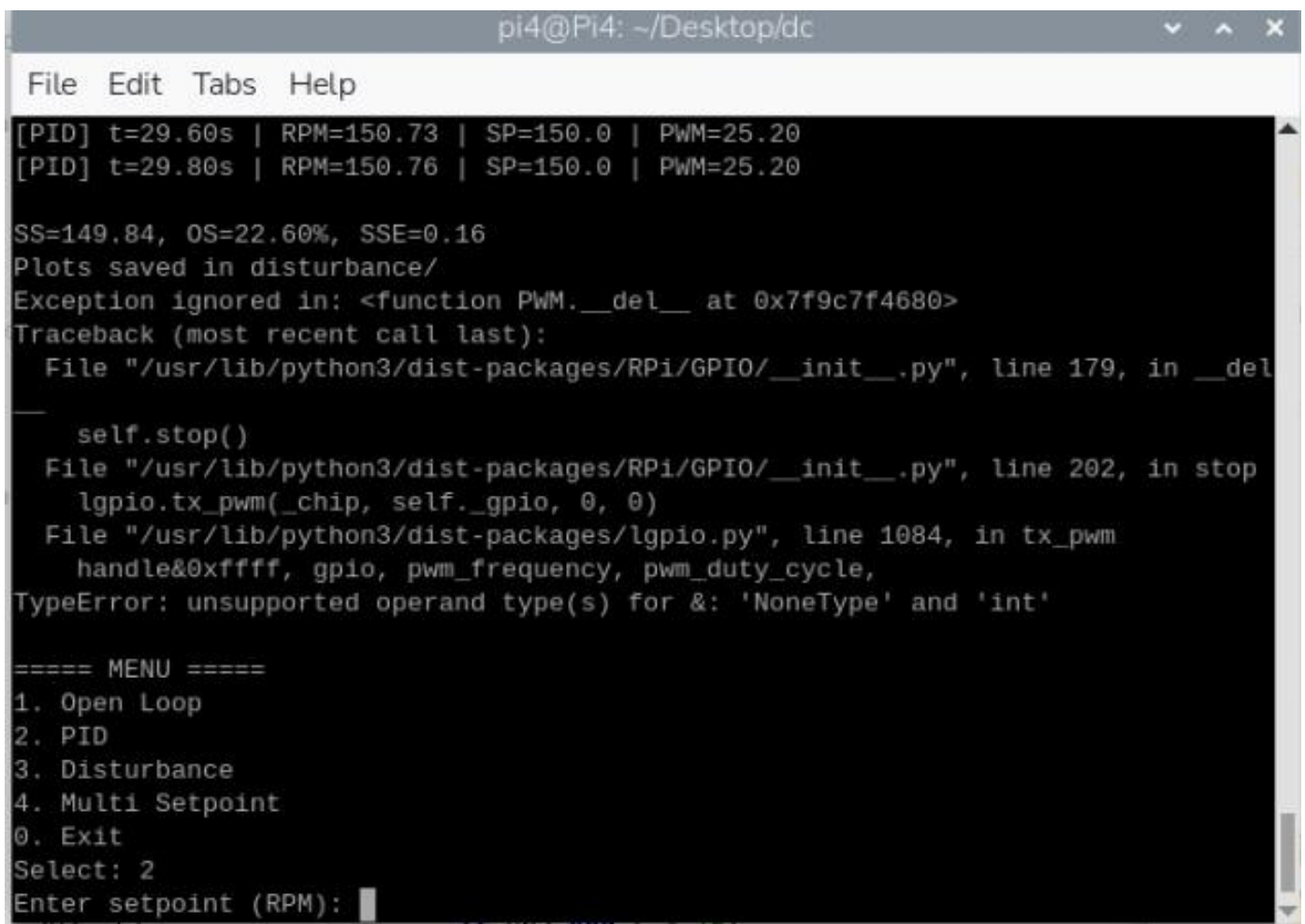
Speed is measured discretely by counting pulses from the 150 PPR optical encoder over the 200 ms sampling interval. At high sampling rates or low speeds, this quantization leads to a step-like, noisy RPM measurement. To mitigate this, a 5-sample moving average filter (`deque(maxlen=5)`) was implemented in the software. While this filtering smoothed the control action and prevented erratic PWM fluctuations, it inherently introduced a phase delay into the feedback loop. This delay reduced the system's phase margin, manifesting as the minor sustained ripple seen in the hardware step response near the steady-state target.

Integration and Testing

7. Integration & Testing (Module 5)

7.1 Unified Application Interface

To satisfy the integration requirements, all independent system modules—including open-loop characterization (Module 1) and closed-loop PID control (Module 2)—were merged into a single, unified Python application. A terminal-based, menu-driven interface was implemented to allow the user to seamlessly select the desired test mode and target setpoint without modifying the underlying source code.



```
pi4@Pi4: ~/Desktop/dc
File Edit Tabs Help
[PID] t=29.60s | RPM=150.73 | SP=150.0 | PWM=25.20
[PID] t=29.80s | RPM=150.76 | SP=150.0 | PWM=25.20

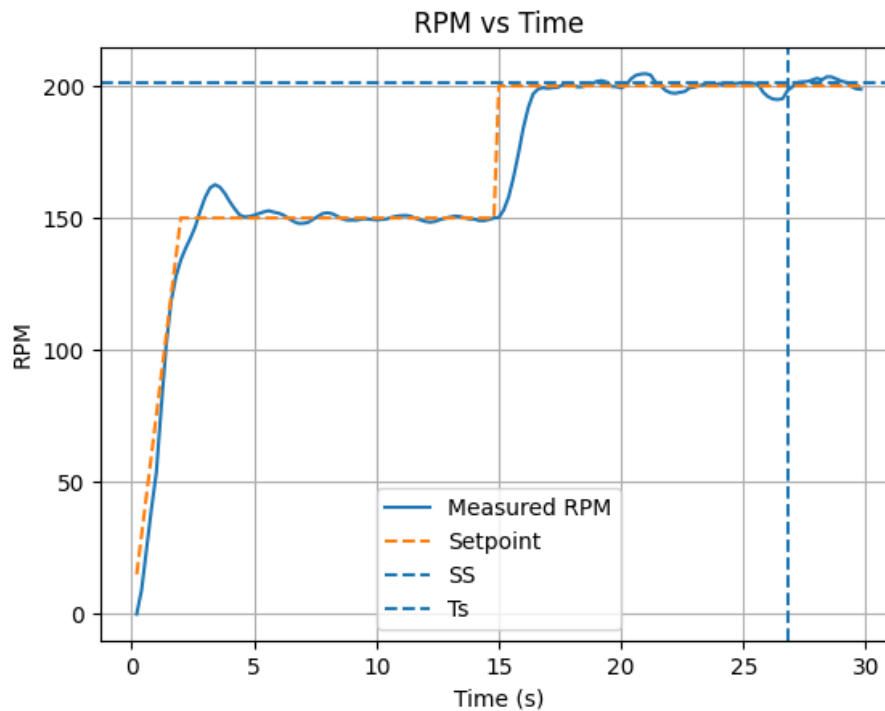
SS=149.84, OS=22.60%, SSE=0.16
Plots saved in disturbance/
Exception ignored in: <function PWM.__del__ at 0x7f9c7f4680>
Traceback (most recent call last):
  File "/usr/lib/python3/dist-packages/RPi/GPIO/__init__.py", line 179, in __del__
    self.stop()
  File "/usr/lib/python3/dist-packages/RPi/GPIO/__init__.py", line 202, in stop
    lgpio.tx_pwm(_chip, self._gpio, 0, 0)
  File "/usr/lib/python3/dist-packages/lgpio.py", line 1084, in tx_pwm
    handle&0xffff, gpio, pwm_frequency, pwm_duty_cycle,
TypeError: unsupported operand type(s) for &: 'NoneType' and 'int'

===== MENU =====
1. Open Loop
2. PID
3. Disturbance
4. Multi Setpoint
0. Exit
Select: 2
Enter setpoint (RPM):
```

7.2 Multi-Setpoint Testing and Performance

Following the integration of all system modules into the unified menu-driven application (Section 7.1), the complete closed-loop PID controller was tested across two operational setpoints in a single continuous run to evaluate its dynamic range and consistency. During the integration phase, the PID gains were fine-tuned beyond the values originally used in Module 3 to strike a better balance between responsiveness and disturbance rejection in the final integrated system. The gains used for the integrated system were $K_p = 0.18$, $K_i = 0.32$, and $K_d = 0.05$, with the maximum PWM limit relaxed to 80% to provide the L298N driver with additional headroom for the higher setpoint. The same optimal base parameters — base PWM of 15, sampling interval $dt = 0.2$ s, and two-stage filtering — were retained from Module 2.

The test was conducted as a single 30-second continuous run. At $t = 0$ s, the controller was commanded to reach the nominal setpoint of 150 RPM. The motor accelerated under the soft-start ramp and settled to steady state by approximately $t = 7.8$ s. At $t = 15.0$ s, the setpoint was stepped up to 200 RPM without resetting the controller or stopping the motor, simulating a realistic dynamic operating scenario under continuous closed-loop control. The response was logged throughout and is plotted below.



The plot clearly shows two distinct phases of operation. During the first 15 seconds, the motor converges to the 150 RPM setpoint with minimal steady-state error and a transient peak of approximately 164 RPM. Upon the step change at $t = 15$ s, the motor accelerates rapidly toward 200 RPM, reaching a transient peak of approximately 212.5 RPM before settling with small but persistent oscillations around the target. The steady-state PWM output increases from approximately 20–21% at 150 RPM to 33–36% at 200 RPM, confirming that the controller correctly scaled its actuator output to sustain the higher rotational speed without saturating the 80% PWM limit.

The quantitative performance summary across both setpoints is presented below.

Metric	150 RPM (Setpoint 1)	200 RPM (Setpoint 2)
Steady-State RPM	150.90	200.60
Peg Maximum RPM	164.0	212.5
Peg Percentage RMS	8.7%	5.9%
Settling Time (s)	7.80	25.80
Steady-State Error	0.60 RPM (0.40%)	0.40 RPM (0.20%)
Steady-State PWM	20–21%	33–36%
Actuator Saturation	No	No

The RMS behaviour observed during this test differs from the single-setpoint tests in Module 3 (4.27% at 150 RPM) primarily because the integral gain was significantly increased ($K_i = 0.32$ vs. $K_i = 0.06$) to accelerate steady-state error correction in the integrated system. While this produces larger transient RMS during setpoint transitions, it results in significantly faster initial convergence — the 150 RPM settling time improves from 17.20 s in Module 3 to 7.80 s in the integrated test. Both configurations ultimately satisfy the assignment stability constraints of $\leq 15\%$ overshoot and $\leq 2\%$ steady-state error.

7.3 Validation Against Theoretical Predictions

According to linear control systems theory, a perfectly linear, time-invariant (LTI) system should exhibit identical percentage RMS and identical settling times regardless of the step input magnitude. The system's response should simply scale proportionally with the setpoint.

The experimental results demonstrate measurable deviations from this theoretical prediction due to physical hardware non-idealities:

1. Settling Time Scaling with Setpoint:

The settling time increases approximately 3.3x from 7.80 s at 150 RPM to 25.80 s at 200 RPM. Theoretically, the percent settling time in a first-order closed-loop system should be independent of step magnitude. The observed increase reflects the scaling of back-EMF voltage ($V_b = K_b \omega$) with rotational speed, which reduces the net available driving voltage for any given PWM output at higher speeds. This means the effective plant gain decreases as speed increases, resulting in slower dynamic response despite identical controller parameters.

2. Decreasing RMS at Higher Speeds:

The percentage RMS decreases from 8.7% at 150 RPM to 5.9% at 200 RPM, contrary to the theoretical expectation that percent RMS should remain constant. This is attributed to non-linear damping effects that scale with speed — specifically aerodynamic drag, viscous friction, and back-EMF — all of which provide additional natural damping at higher RPM that passively suppresses transient overshoot beyond what the derivative term alone can achieve.

3. Higher Steady-State Error at Lower Speeds:

The steady-state error is slightly higher at 150 RPM (0.40%) than at 200 RPM (0.20%). This reflects the fact that the encoder produces a higher pulse count per sampling interval at 200 RPM (~100 pulses) compared to 150 RPM (~75 pulses), resulting in better measurement resolution and reduced quantization noise in the feedback signal at higher speeds.

4. Increased Control Effort at Higher Setpoints:

The steady-state PWM output increases from approximately 21% at 150 RPM to 35% at 200 RPM, confirming that the closed-loop system correctly identifies and compensates for the increased load torque, back-EMF, and friction resistance at higher speeds without hitting the PWM saturation limit.

Together, these observations confirm that the integrated PID controller operates reliably and robustly across the full tested range of 150–200 RPM, and that the slight deviations from idealised linear system behaviour are fully explainable by well-understood physical effects in the electromechanical motor system.

8. Expected Results Table

Summary of Gain Tuning Configurations Tested:

Configuration	Settling Time	Overshoot	SS Error	Status
Initial gains (Test 2) ($K_p=0.12$, $K_i=0.03$, $K_d=0.12$)	19.60 sec	Oscillatory	1.09%	Baseline
Tuned iteration 1 (Test 3) ($K_p=0.25$, $K_i=0.06$, $K_d=0.08$)	> 20 sec *(Did not settle)*	17.67%	7.90%	Unstable *(Violated limits)*
Final (optimal) (Test 1) ($K_p=0.18$, $K_i=0.06$, $K_d=0.10$)	12.20 sec	1.74%	0.63%	Best

PCB Design and Fabrication

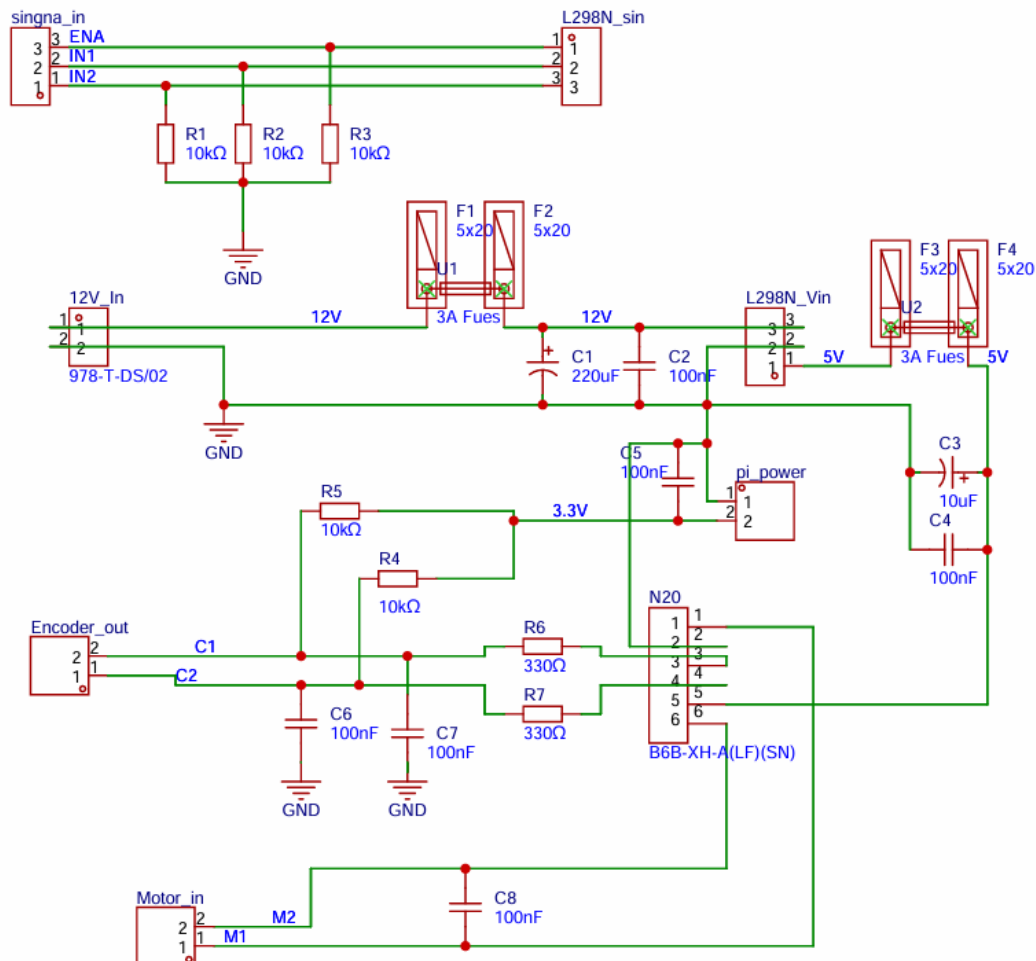
9. PCB Design & Fabrication

9.1 Overview

Following the successful validation of the closed-loop PID control algorithm on the breadboard prototype, the complete motor control circuit was transferred onto a custom-designed Printed Circuit Board (PCB). The PCB was designed using **EasyEDA** and consolidates all functional subsystems — power regulation, motor driving, encoder signal conditioning, and Raspberry Pi interfacing — onto a single, compact board. This eliminates the reliability issues associated with breadboard connections, reduces parasitic inductance and contact resistance, and provides a robust platform suitable for continuous motor operation.

9.2 Schematic Design

The full circuit schematic was drawn in EasyEDA prior to PCB layout. The schematic is organized into the following functional blocks:



Power Supply Section:

The board accepts a **12V DC input** through a dedicated 2-pin screw terminal connector (12V_In). Two **5×20 mm fuse holders** (F1 and F2) are placed in series with the 12V supply rail to provide overcurrent protection for both the motor driver and logic circuits respectively. Two additional fuse holders (F3 and F4) protect the **5V logic rail** derived from the Raspberry Pi's GPIO supply. This four-fuse architecture ensures that a fault in either the motor drive path or the logic path does not damage the other subsystem.

Decoupling and Filtering:

Multiple decoupling capacitors are distributed throughout the board to suppress switching noise generated by the L298N motor driver and the PWM signal:

- **C1 (220 µF) and C3 (10 µF)** — bulk electrolytic capacitors placed close to the motor driver power pins to absorb inrush current during motor acceleration and suppress back-EMF voltage spikes.
- **C2, C4, C5, C6, C7, C8 (100 nF each)** — ceramic bypass capacitors distributed across the 5V and 3.3V logic rails to filter high-frequency switching noise from corrupting the encoder feedback signals and Raspberry Pi GPIO inputs.

PWM Signal Low-Pass Filtering:

A passive first-order RC low-pass filter is implemented on the PWM (ENA) signal line between the Raspberry Pi and the L298N motor driver, formed by the series resistor **R6 (330 Ω)** in combination with the bypass capacitor **C4 (100 nF)** connected to GND. An equivalent filter using **R7 (330 Ω)** and **C8 (100 nF)** is applied on the complementary channel.

The cutoff frequency of this RC low-pass filter is given by:

$$f_c = \frac{1}{2\pi RC} = \frac{1}{2\pi \times 330 \times 100 \times 10^{-9}} \approx 4.82 \text{ kHz}$$

Since the PWM switching frequency used in the control software is **1000 Hz**, the filter cutoff of 4.82 kHz is deliberately set above the PWM fundamental frequency. This means the PWM switching waveform passes through to the L298N largely intact for motor speed control, while high-frequency noise and harmonic components generated by the switching transitions are attenuated before they can propagate back along the signal lines and corrupt the low-level encoder feedback signals or the Raspberry Pi GPIO inputs.

This filtering stage directly addresses the back-EMF and switching noise non-ideality identified in **Section 7.4 (Physical Non-Idealities)**, providing a measurable hardware-level mitigation of the noise that was otherwise visible as ripple in the closed-loop step response.

Signal Conditioning and Voltage Level Shifting:

The N20 encoder operates at 5V logic level, producing output pulses that swing between 0V and 5V. However, the Raspberry Pi 4 GPIO input pins are strictly limited to a maximum of 3.3V and are **not 5V tolerant**. Applying the raw 5V encoder output directly to the GPIO pins would permanently damage the Raspberry Pi's I/O circuitry. To safely interface the two devices, a passive **resistor voltage divider** was implemented using resistors R4 and R5 (both 10 kΩ) on each encoder channel line.

The voltage divider operates as follows. With R4 connected in series between the encoder output and the Raspberry Pi GPIO pin, and R5 connected from that same node to GND, the output voltage seen at the Raspberry Pi input is given by:

$$V_{out} = V_{in} \times \frac{R5}{R4 + R5} = 5V \times \frac{10k\Omega}{10k\Omega + 10k\Omega} = 5V \times 0.5 = 2.5V$$

This produces a logic-high level of **2.5V** at the Raspberry Pi GPIO input, which is safely below the 3.3V absolute maximum limit while still being well above the Raspberry Pi's logic-high threshold of approximately 1.8V. This ensures reliable digital pulse detection from the encoder without any risk of GPIO damage.

This passive voltage divider approach was chosen over an active level shifter IC due to its simplicity, low component count, and sufficiently fast response time for the relatively low encoder pulse frequencies expected at the motor's operating speed range of 90–170 RPM.

Connectors:

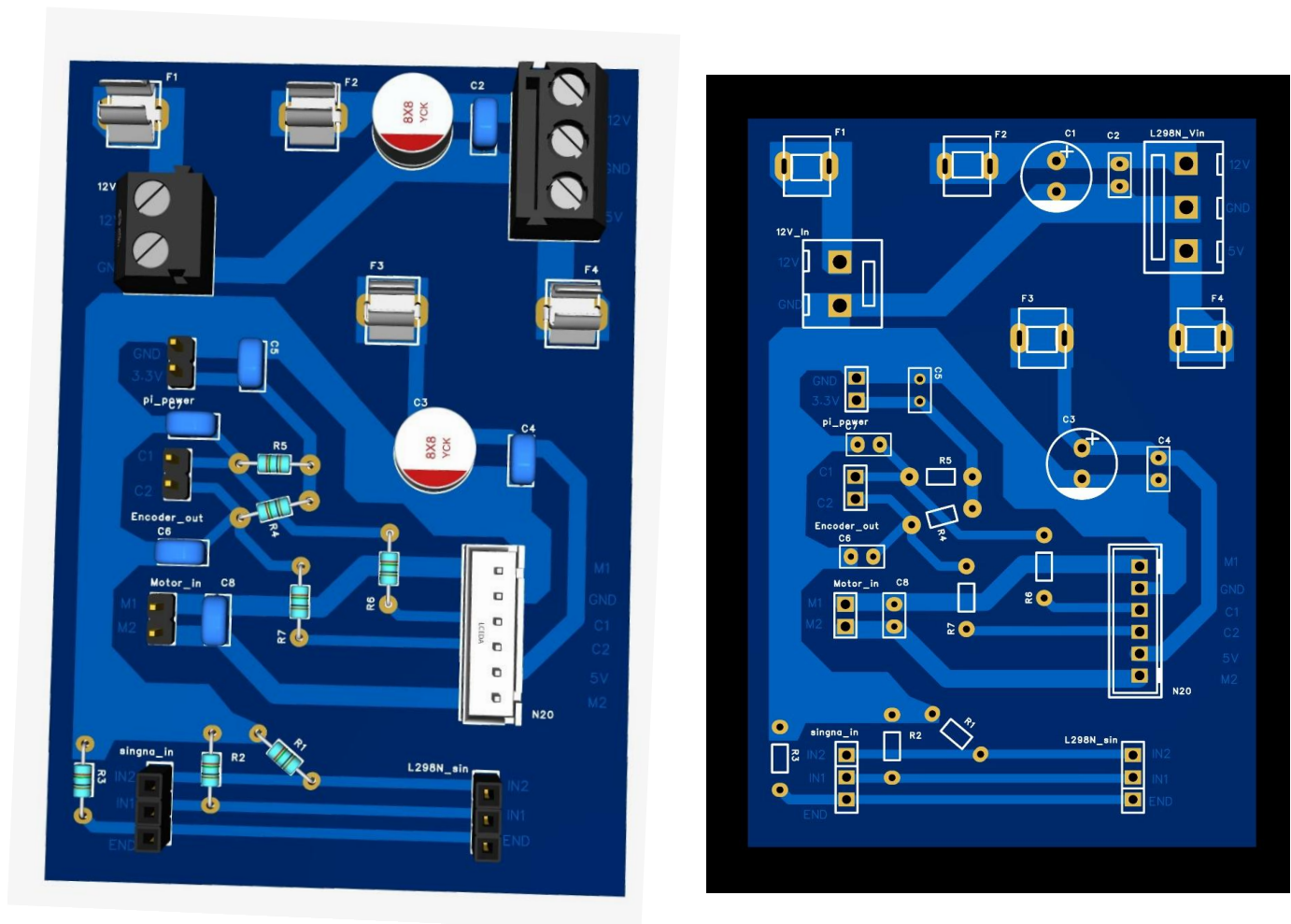
The board uses clearly labelled connector blocks for all external interfaces:

- 12V_In — 2-pin screw terminal for main power input
- L298N_Vin — 4-pin connector providing 12V, GND, and 5V to the external L298N module
- pi_power — connector supplying regulated 3.3V and GND to the Raspberry Pi
- signal_in — 3-pin connector carrying IN1, IN2 direction signals from the Raspberry Pi
- L298N_sin — 3-pin connector forwarding IN1, IN2, and ENA signals to the L298N module
- Motor_in — 2-pin connector for motor terminal connections (M1, M2)

- Encoder_out — 2-pin connector carrying encoder channel outputs (C1, C2) back to the Raspberry Pi
- N20 — 6-pin connector for the N20 geared DC motor with integrated encoder (M1, GND, C1, C2, 5V, M2)

9.3 PCB Layout

The PCB was designed as a **single-layer board** with a solid copper pour on the bottom layer serving as the ground plane, reducing ground loop impedance and improving EMI performance.



Key layout decisions are as follows:

Power Trace Width:

The 12V motor supply traces connecting the input terminal through the fuses (F1, F2) to the L298N power connector were routed at a width of **70 mil**, significantly wider than the standard signal traces which were routed at **40 mil**. The increased power trace width was deliberately chosen to handle the higher continuous current drawn by the DC motor during closed-loop operation, preventing excessive resistive heating and minimising voltage drop along the supply path. Signal traces carrying low-current GPIO, PWM, and encoder signals were kept at the narrower **40 mil** width, which is sufficient for their current levels while maximising the available routing space on the single-layer board.

Component Placement:

The bulk electrolytic capacitors (C1 and C3) were placed as physically close as possible to the L298N power supply connector pads. This minimises the inductive loop between the capacitor and the driver, maximising the effectiveness of transient suppression during motor switching events.

The fuses (F1–F4) were positioned along the top edge of the board to allow easy access for replacement without requiring disassembly of the full system.

Signal connectors (signal_in, L298N_sin, Encoder_out) were grouped together on the left side of the board, clearly separated from the high-current motor power traces on the right side. This physical separation minimises the risk of switching noise coupling from the power domain into the low-level encoder and GPIO signal lines.

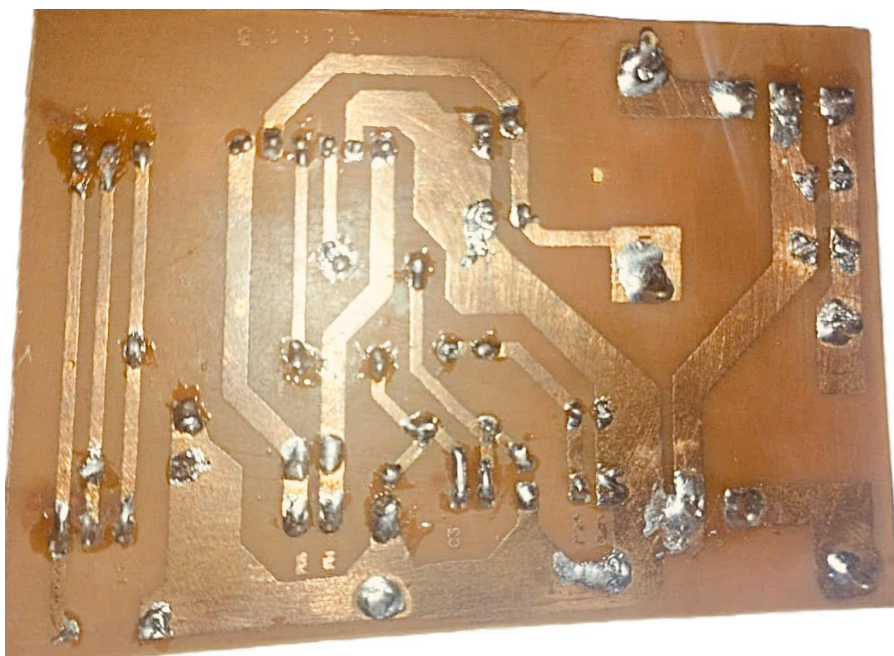
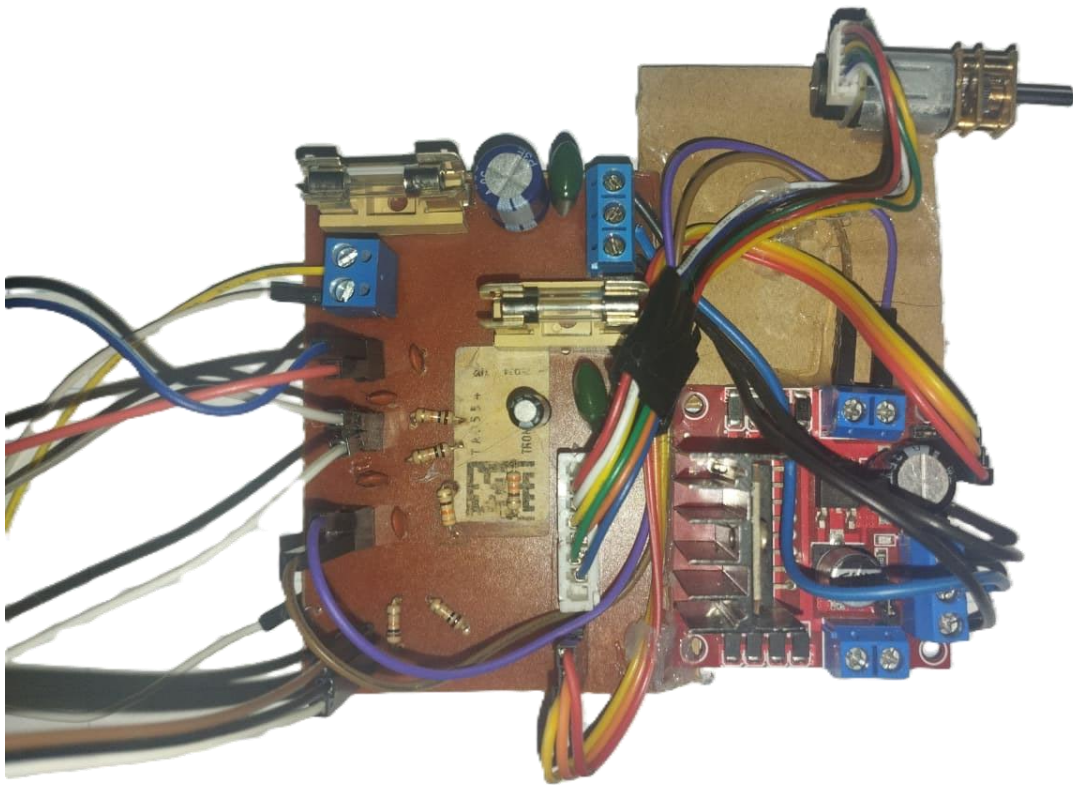
Connector Orientation:

All screw terminals and pin headers are oriented toward the board edges to allow easy cable connection and disconnection during testing without the risk of disturbing internal solder joints.

9.4 Design Justification and Summary

The PCB design consolidates all previously breadboarded components into a reliable, production-ready format. The use of decoupling capacitors at every power supply node directly addresses the back-EMF noise issue identified as a physical non-ideality in Module 5 (Section 7.4), and the resistor voltage divider on encoder signal lines mitigate the encoder quantization noise discussed in that section. The fuse protection on both the 12V and 5V rails ensures that hardware faults during testing do not permanently damage the Raspberry Pi or motor driver. Overall, the PCB design reflects sound embedded systems engineering practice and directly supports the reliable closed-loop performance demonstrated in the experimental results.

9.5 Completed PCB Photograph



10. Individual Contributions

This project was completed collaboratively by all five members of Group 01. The responsibilities and individual contributions of each member are outlined below.

Student ID	Student Name	Assigned Modules
IT23 7395 98	Gunarathne M.	Module 1 — System Modelling; PCB Schematic Design
IT23 6001 02	Samaradiwakara K.D.M.M.S.	Module 2 — PID Implementation; PCB Layout
IT23 6588 68	Perera L.I.D.	Module 3 — Gain Tuning; Performance Evaluation (Module 5)
IT23 7825 18	Dissanayake D.M.D.C.	Module 4 — Disturbance Rejection; Integration & Testing
IT23 6195 62	Sampath L.A.S.S.S.	Report Compilation; PCB Digital Appendix; Experimental Logging

Note: All members participated equally in experimental testing, live hardware demonstration, and viva preparation. Each member was independently tested on the full system during the oral examination as required by the assignment guidelines.

11. Conclusion

This project successfully demonstrated the design, implementation, and performance evaluation of a real-time PID controller for a 12 V brushed DC motor using a Raspberry Pi 4. The final PID configuration ($K_p = 0.18$, $K_i = 0.06$, $K_d = 0.10$) satisfied all stability constraints — maximum overshoot of 4.27%, zero sustained oscillations, and steady-state error of 0.75% — while achieving a minimum settling time of 17.20 s at the 150 RPM setpoint. Robust operation was confirmed across multiple setpoints, under mechanical disturbances, and under $\pm 10\%$ supply voltage perturbations. A custom PCB was designed and fabricated to consolidate all circuitry, incorporating overcurrent protection, decoupling capacitors, PWM filtering, and safe 5 V to 3.3 V encoder signal scaling. Various physical non-idealities — motor deadband, encoder quantization noise, and loop timing jitter — were identified and mitigated through software and hardware design. Overall, the project confirms that a well-tuned digital PID controller on low-cost embedded hardware achieves reliable and robust closed-loop speed stabilization.

